URBAN SUSTAINABILITY CODE-LEVEL BLUEPRINT

A code-level build specification for an AI-driven sustainability decision engine for Indian cities.

Aligned to the supplied requirements: environmental impact, economic feasibility, social equity, limited budgets and land, climate uncertainty, five MVP modules, and the listed AWS stack.

Scope: The attached document is a product requirements brief, not prototype source code. This blueprint translates its requirements into a proposed implementation; it does not claim to recover existing code. Examples and values are illustrative.

Prepared 23 October 2026

1. Requirements translated into build behavior

The supplied Sustainable Development brief defines the goal as choosing intervention combinations for Indian cities while balancing environment, economic feasibility, and social equity. It calls out urban heat, flooding, water scarcity, carbon emissions, budgets, land availability, climate uncertainty, and five MVP components. The following implementation mapping follows that brief.

The five required MVP components

Brief component	Code-level responsibility	Demo output
Data Engine	Ingest and validate climate, water, land-use, and population layers; retain source, timestamp, spatial unit, and coverage.	Neighborhood baseline with data-quality flags.
AI Optimisation	Generate feasible packages of green roofs, urban forests, rainwater harvesting, and drainage improvements.	Ranked packages under budget and land constraints.
Interactive Dashboard	Map risks, costs, resource allocation, and who benefits at neighborhood scale.	Map + comparison cards + equity view.
Scenario Simulator	Recalculate outcomes under drought, extreme rainfall, and sudden budget reduction.	Baseline vs stress-case trade-offs.
Decision Explainer	Explain score contributions, uncertainty, beneficiaries, and possible unintended consequences.	Auditable why-this-option panel.

References from the earlier comparison: ArcGIS Urban and URBANopt may remain useful product/technical references, but they are not named in this requirements brief. The required deployment stack here is AWS.

2. Proposed five-module architecture

1. Data and domain models. Places, indicators, interventions, constraints, and scenario inputs/outputs. Normalize units; retain provenance.

2. Option generation and optimization. Build feasible intervention bundles, calculate transparent scores, return ranked candidates and trade-offs.

3. Scenario simulation. Apply selected interventions to a baseline under explicit assumptions; return estimates and sensitivity ranges.

4. Dashboard and comparison. Present baseline vs scenario, indicator cards, map/list views, filters, comparison charts.

5. Decision explainer. Report ranking drivers, constraints, assumptions, and data gaps.

Request flow

User selects place and baseline -> validate data coverage -> generate candidates -> simulate outcomes -> compare in dashboard -> explain drivers and caveats. Save inputs and model version to reproduce a result.

Example data contracts

```
from datetime import datetime
from typing import Literal
from pydantic import BaseModel, Field

class Indicator(BaseModel):
    key: str
    value: float
    unit: str
    direction: Literal["higher_is_better", "lower_is_better"]
    source: str
    observed_at: datetime
    confidence: float = Field(ge=0, le=1)

class Intervention(BaseModel):
    id: str
    name: str
    cost: float = Field(ge=0)
    effects: dict[str, float]
    prerequisites: list[str] = []
```

Example values are mock data. Keep raw and normalized values, units, source, timestamp, and confidence separate.

3. Optimization and scenario simulation

A weighted score can support a transparent hackathon shortlist if normalization, metric direction, constraints, and user priorities are explicit. It does not resolve every planning trade-off.

```
def rank_score(benefits, priorities):
    weights = {k: max(0.0, v) for k, v in priorities.items()}
    total = sum(weights.values())
    if total == 0:
        raise ValueError("At least one positive priority is required")
    return sum(benefits.get(k, 0.0) * w
               for k, w in weights.items()) / total

def feasible(bundle, budget):
    return sum(x.cost for x in bundle) <= budget

def simulate(baseline, interventions):
    estimate = dict(baseline)
    for item in interventions:
        for key, delta in item.effects.items():
            estimate[key] = estimate.get(key, 0.0) + delta
    return estimate  # additive toy model; not a validated forecast
```

Safeguards

- Normalize metrics using documented bounds; do not compare raw units.
- Keep hard constraints separate from preferences and show user weights.
- Return a Pareto shortlist when criteria conflict.
- Do not silently treat missing indicators as zero benefit.
- Label results estimates for decision support, not guaranteed recommendations.

Simulation limits

Additive effects can double-count benefits when interventions interact. Real effects may depend on location, season, maintenance, and climate. A later model could add spatial exposure, interaction terms, or calibrated domain models. Show limitations instead of implying scientific precision.

Reproducibility

Persist baseline snapshot or dataset version, selected interventions, weights, constraints, model version, run timestamp, and outputs.

4. Dashboard and decision explainer

Use consistent definitions, units, and horizons for baseline and scenario. Provide a table alternative to map-only information. Show uncertainty and data gaps alongside benefits.

Example dashboard payload

```
{
  "scenario_id": "demo-001",
  "baseline": {"heat_exposure": 0.68, "canopy_pct": 12},
  "estimate": {"heat_exposure": 0.57, "canopy_pct": 16},
  "units": {"heat_exposure": "index 0..1", "canopy_pct": "%"},
  "cost": 420000,
  "constraints_met": true,
  "data_gaps": ["summer pedestrian counts"]
}
```

Decision explanation example

```
def explain(score_parts, constraints, assumptions, gaps):
    drivers = sorted(score_parts.items(), key=lambda x: x[1], reverse=True)
    return {
        "top_drivers": drivers[:3],
        "constraints_met": constraints,
        "assumptions": assumptions,
        "data_gaps": gaps,
        "caution": "Estimated effects; validate locally."
    }
```

Dashboard checklist

Show baseline and selected scenario together; state units, source, and direction of improvement; show cost, feasibility, confidence/data coverage; make priorities inspectable; explain top score drivers and missing data; use accessible labels and contrast.

> Do not call one score “sustainability” without naming its dimensions. A composite score is a chosen summary, not objective truth.

5. Implementation next step

Build one end-to-end slice for one study area and two interventions. This will test the data contracts, comparison view, and explanation before the team adds more indicators or advanced solvers.

1. Choose a place and baseline. Use one neighborhood or small synthetic grid; record source and date for each measure.

2. Define two interventions. For example, shade trees and cool roofs; mark costs and effects as illustrative.

3. Implement the contracts. Typed models, feasibility check, ranking, simulation, explanation output.

4. Render a comparison. Show a few indicators, cost, constraints, data gaps, and why an option ranked as shown.

5. Review with a domain mentor. Replace mock assumptions with local evidence before presenting decision-support claims.

Definition of done

A user selects a baseline, sets priorities and a budget, compares at least two feasible scenarios, inspects assumptions and sources, and understands why the displayed option ranked higher. Inputs are saved for reproducibility and outputs include a plain-language caveat.

Suggested project layout

```
app/domain/models.py
app/services/ranking.py
app/services/simulation.py
app/services/explanations.py
app/api/scenarios.py
app/ui/dashboard.py
data/demo_baseline.json
data/demo_interventions.json
README.md  # assumptions, sources, limitations
```

6. Deeper code-level build specification

This section turns the feature list into implementable data flow and module contracts. The architecture can be built as one FastAPI application plus a React map/dashboard. Keep domain logic separate from UI so the same calculations power the dashboard and exported brief.

Repository structure

```
backend/app/
  domain/models.py          typed input/output contracts
  domain/metrics.py         normalization and direction rules
  services/catalog.py       intervention catalog + validation
  services/planner.py       feasible package generation
  services/scoring.py       weighted score + Pareto filtering
  services/simulation.py    baseline-to-scenario estimates
  services/explanations.py  human-readable ranking rationale
  api/routes_scenarios.py   request validation and persistence
frontend/src/
  MapPanel.tsx              area selection + indicator overlays
  ScenarioBuilder.tsx       intervention and budget controls
  ComparisonPanel.tsx       baseline/scenario charts
  WhyThisRanked.tsx         drivers, assumptions, missing data
  BriefExport.tsx           print/shareable decision summary
```

Core module boundaries

Catalog owns intervention definitions, allowed units, cost ranges, required inputs, and effect assumptions. **Planner** only assembles feasible combinations. **Scoring** orders options from normalized outcome values and user weights. **Simulation** evaluates the selected package against one frozen baseline. **Explanation** consumes saved results; it never recalculates or invents an outcome.

Every scenario result should include scenario_id, baseline_version, intervention_ids, priority_weights, constraints, metric_results, score_breakdown, assumptions, data_gaps, model_version, and created_at.

7. Detailed contracts and scoring

Represent each intervention's effects as explicit estimates with units, geography, evidence note, and confidence. A single effect dictionary is convenient for a demo but hides important metadata; the richer model below keeps it auditable.

```
from pydantic import BaseModel, Field
from typing import Literal

class Effect(BaseModel):
    metric: str
    delta: float
    unit: str
    evidence_note: str
    confidence: float = Field(ge=0, le=1)

class Intervention(BaseModel):
    id: str
    label: str
    cost: float = Field(ge=0)
    land_m2: float = Field(ge=0)
    group_benefits: dict[str, float] = {}
    effects: list[Effect]
    requires: list[str] = []
    excludes: list[str] = []
    min_area_m2: float = 0

class ScenarioRequest(BaseModel):
    area_id: str
    baseline_version: str
    budget: float = Field(ge=0)
    priorities: dict[str, float]
    required_interventions: list[str] = []
    max_options: int = Field(default=8, ge=2, le=20)
```

Normalize by goal, not by raw number

Convert each metric change to a benefit score in [0,1] using published or demo-defined bounds. For "lower is better", reverse the direction. Show the bounds in the UI and avoid changing them silently between runs.

```
def benefit_score(value, low, high, direction):
    if high <= low:
        raise ValueError("normalization bounds must increase")
    x = min(1.0, max(0.0, (value - low) / (high - low)))
    return x if direction == "higher" else 1.0 - x

def weighted_score(metric_benefits, weights):
    active = {k: max(0.0, weights.get(k, 0.0))
              for k in metric_benefits}
    total = sum(active.values())
    if total == 0:
        raise ValueError("choose at least one priority")
    parts = {k: metric_benefits[k] * w / total
             for k, w in active.items()}
    return sum(parts.values()), parts
```

The score breakdown is a first-class output: each metric's contribution should be available to the explanation panel. Keep feasibility (budget, exclusions, minimum area) as a pass/fail filter before ranking, not as a hidden penalty.

8. Candidate generation, what-if, and explanations

For a hackathon catalog of perhaps 5–10 interventions, exhaustive combinations are practical. Use exhaustive enumeration first because it is inspectable and easy to debug. If the catalog grows, replace it with a solver while keeping the same output contract.

```
from itertools import combinations

def generate_packages(catalog, budget, required=(), max_size=4):
    required = set(required)
    for size in range(1, min(max_size, len(catalog)) + 1):
        for bundle in combinations(catalog, size):
            ids = {x.id for x in bundle}
            if not required <= ids:
                continue
            if sum(x.cost for x in bundle) > budget:
                continue
            if any(set(x.excludes) & ids for x in bundle):
                continue
            if any(not set(x.requires) <= ids for x in bundle):
                continue
            yield bundle
```

Pareto filtering

A package is dominated if another package is at least as good on every selected metric and strictly better on one. Keep nondominated choices available as alternatives so a single weight vector does not erase useful trade-offs.

```
def dominates(a, b, keys):
    no_worse = all(a[k] >= b[k] for k in keys)
    strictly_better = any(a[k] > b[k] for k in keys)
    return no_worse and strictly_better

def pareto_front(options, keys):
    return [a for a in options
            if not any(b is not a and dominates(b, a, keys)
                       for b in options)]
```

Live what-if behavior

The budget slider and priority controls should update a local request state, debounce recalculation, and return a new run_id. Cache results by a hash of area, baseline version, intervention catalog version, budget, and weights. Never mutate a saved scenario in place; create a new run so judges can see what changed.

Explanation output should be deterministic

```
{
  "headline": "Package B ranks highest for these priorities",
  "drivers": [{"metric":"heat","contribution":0.31},
              {"metric":"canopy","contribution":0.24}],
  "tradeoffs": ["higher upfront cost than Package A"],
  "constraints": {"budget_met": true},
  "assumptions": ["tree survival estimate: demo value"],
  "data_gaps": ["no block-level shade observation"]
}
```

9. API, screen behavior, and winning demo

API contracts keep the UI straightforward and make the demo reproducible. Validate all user input on the server; return field-level errors rather than silently adjusting values.

```
POST /api/v1/runs
  body: ScenarioRequest
  returns: RunResult with ranked options and explanations

GET /api/v1/runs/{run_id}
  returns: immutable saved inputs + outputs

GET /api/v1/areas/{area_id}/baseline
  returns: metrics, source links, coverage, version

GET /api/v1/interventions?area_id=...
  returns: eligible interventions and assumptions
```

Screen-by-screen behavior

1. Select area: map click or search; show baseline date, metric coverage, source freshness, and a missing-data notice.
2. Build package: choose interventions with cost, expected metric direction, prerequisites, and evidence note; show budget remaining.
3. Compare: baseline plus two or three alternatives, with consistent units, absolute changes, costs, and confidence/coverage.
4. Explain: show top score contributions, trade-offs, constraints, and assumptions in plain language.
5. Share: create a printable brief from the exact saved run, with source links and caveat.

Winning hackathon narrative

Tell one concrete story: “This Indian neighborhood faces heat and water/flood risk. Under a stated budget and land limit, these intervention packages compare environmental gains, cost, and which groups benefit. Here is why one package fits the chosen priorities, where the estimate is uncertain, and what evidence would improve the decision.” A credible explanation and polished end-to-end flow are stronger than many disconnected features.

Demo must-haves

Use a preloaded area and curated interventions so the demo works offline; ensure every visible metric has units and a source; make controls update results quickly; show one clear limitation; prepare a fallback screenshot or saved run; end on the shareable brief. Do not claim measured emissions or health impacts unless the underlying data and method support them.

Acceptance criteria for the slice

Area selection loads a versioned baseline; invalid budget/weights produce clear errors; infeasible bundles never rank; changing budget or weights creates a new reproducible result; each score has metric contributions; missing inputs remain visible; exported brief matches the saved run.

10. Source alignment and evidence

This blueprint maps to the supplied Sustainable Development requirements brief: an AI-driven urban sustainability decision engine for Indian cities, balancing environmental impact, economic feasibility, and social equity; the five required MVP components; climate disruption and budget-cut scenarios; and the listed AWS services. The brief does not include prototype source code, schemas, or calculation formulas, so these implementation details are proposed rather than recovered.

Earlier comparison references

ArcGIS Urban and URBANopt appeared in the prior discussion and remain contextual references, not requirements stated in the supplied brief. The requested technology mapping below follows the brief's AWS stack. Greenopt is not identified in the supplied document; no claims are attributed to it.

AWS documentation used to ground service roles

Amazon S3, Glue, and Athena: Athena queries data in S3 with SQL; Glue Data Catalog supplies table metadata and crawlers can infer schemas.
Amazon Athena
Glue Data Catalog with Athena

SageMaker AI: AWS documents managed training jobs and deployment/inference workflows. For this MVP it can host a learned risk/forecast model if suitable local training data exists; it should not be used to conceal policy weights or replace explicit constraints.
SageMaker AI model training

SimSpace Weaver: AWS describes it as a service for large-scale spatial simulations. It may be appropriate for a later detailed spatial model, while the hackathon MVP can run transparent aggregate stress calculations in Lambda first.
SimSpace Weaver User Guide

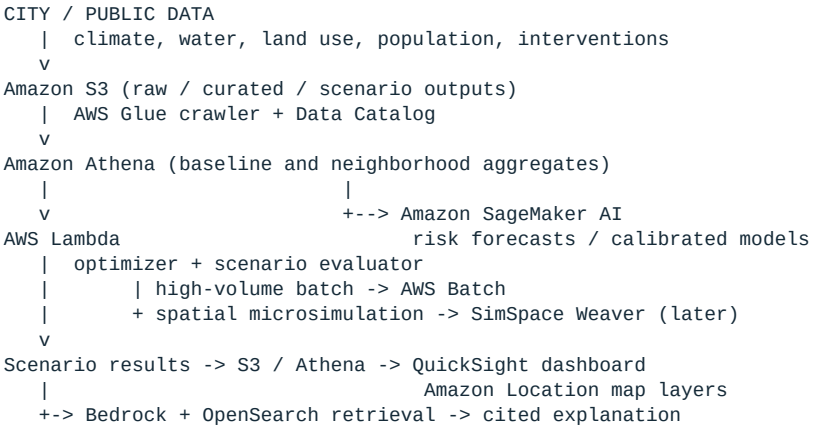
Bedrock, OpenSearch, Location, and QuickSight: use retrieval-grounded text explanations with citations, searchable evidence, map layers, and dashboard views. Bedrock should phrase validated calculation outputs, not invent impacts.
Bedrock Knowledge Bases
Amazon Location Service
Amazon QuickSight dashboards

Remaining validation

A working prototype would still need review against local climate and census data, intervention cost assumptions, land constraints, and feedback from planners and affected communities. Validate calibration and group-level effects before describing outputs as predictions.

11. AWS architecture mapped to the brief

Use the AWS services named in the brief as a modular data and application path. For a hackathon, implement the smallest vertical slice first: S3/Glue/Athena for a curated baseline, Lambda for deterministic optimization and stress calculations, Location Service for map context, and QuickSight or a lightweight web dashboard for the user flow. Add model training, Bedrock, OpenSearch, Batch, EventBridge, and SimSpace Weaver only where they support a specific tested need.



Service responsibility table

Brief module	Named AWS services	Recommended MVP responsibility
Data Engine	S3, Glue, Athena	Store raw/curated layers; catalog schemas; query neighborhood indicators.
AI Optimisation	SageMaker, Lambda, Batch	Lambda applies explicit constraints and ranks packages; SageMaker only for validated forecasts; Batch for larger search.
Scenario Simulator	SimSpace Weaver, Lambda, EventBridge	Lambda computes drought/rain/budget stress runs; EventBridge schedules; Weaver reserved for large spatial simulation.
Decision Explainer	Bedrock, OpenSearch	Retrieve approved evidence and phrase the deterministic score/uncertainty with citations.
Interactive Dashboard	Location Service, QuickSight, Managed Grafana	Location supports map context; QuickSight decision dashboards; Grafana optional for operational time-series monitoring.

Keep responsibilities separated: use deterministic code for feasibility, objective calculations, scenario arithmetic, and ranking. Use generative AI only to explain those saved results from retrieved, approved evidence.

12. Equity-aware optimization and climate shocks

The brief says the engine should show who benefits, not only total citywide change. For the MVP, define a small set of neighborhood-level groups or vulnerability bands using an approved dataset; aggregate results to avoid exposing personal data. Let planners set an equity floor or use a stated equity weight, and expose the choice in the UI.

A transparent multi-objective package score

Normalize each outcome to 0..1 benefit using documented local or demo bounds. Then combine environmental, economic, and equity dimensions with visible weights. Budget, land, and required-service coverage remain hard constraints.

```
def package_score(env, economic, equity, weights):
    # weights are user-visible, nonnegative, and normalized here
    dims = {"environment": env, "economic": economic, "equity": equity}
    w = {k: max(0.0, weights.get(k, 0.0)) for k in dims}
    total = sum(w.values())
    if total == 0:
        raise ValueError("set at least one objective weight")
    parts = {k: dims[k] * w[k] / total for k in dims}
    return sum(parts.values()), parts

def satisfies_constraints(package, budget, land_limit, min_group_gain):
    return (package.cost <= budget
            and package.land <= land_limit
            and all(gain >= min_group_gain
                    for gain in package.group_benefits.values()))
```

Drought, extreme rainfall, and sudden budget cuts

Treat each disruption as a named scenario configuration, not a new opaque model. Drought can reduce available water and raise the value of harvesting/storage; extreme rainfall can increase runoff load and test drainage capacity; a budget cut changes the feasible set. Apply the same package to each stress case and show which packages remain feasible.

```
def simulate_metrics(inputs, package):
    # Baseline and deltas must use matching metric keys and units.
    out = dict(inputs["baseline_metrics"])
    for effect in package.metric_effects:
        out[effect.metric] = out.get(effect.metric, 0.0) + effect.delta
    return out

def stress_run(base_inputs, package, shock):
    inputs = dict(base_inputs)
    inputs.update(shock.overrides)  # e.g. rain multiplier, water supply
    budget = min(inputs["budget"], shock.budget_cap)
    if package.cost > budget:
        return {"feasible": False, "reason": "budget cut"}
    return {
        "feasible": True,
        "metrics": simulate_metrics(inputs, package),
        "shock_id": shock.id,
        "assumptions": shock.assumptions,
    }
```

Equity and uncertainty output

For each scenario, return group-level benefit deltas, the lowest-benefit group, the share of neighborhoods with missing inputs, and a confidence band or evidence grade. Label scenario ranges as sensitivity ranges unless statistically calibrated. Add an unintended-consequence checklist: water demand, displacement risk, maintenance burden, and unequal access.

Hackathon proof point

Demonstrate one neighborhood, two or three intervention packages, three shocks (drought, extreme rainfall, budget cut), and a visible environmental/economic/equity trade-off. A judge should be able to change one priority and see why the recommended package changes.

13. Build order and acceptance checks

Build the demo in this order so every AWS service supports a visible feature instead of becoming setup work without user value.

Slice 1 - Data: one city/neighborhood data bundle in S3, documented schema, Glue catalog, Athena query that yields baseline indicators and coverage flags.

Slice 2 - Decision logic: intervention catalog, cost/land constraints, environmental/economic/equity scores, and one deterministic ranked result in Lambda.

Slice 3 - Shocks: drought, extreme rainfall, and budget cut inputs; show which packages remain feasible and how the metric trade-offs shift.

Slice 4 - Experience: map and dashboard; who-benefits panel; cited explanation generated from saved result; export/share decision brief.

Slice 5 - AI and scale, only if time remains: SageMaker forecast if valid data exists; Bedrock/OpenSearch grounded narrative; AWS Batch or SimSpace Weaver only if package search or spatial model exceeds the MVP.

Acceptance checks

- Each baseline indicator has source, date, unit, spatial level, and coverage.
- A package exceeding budget or land limit is rejected and its reason is shown.
- Users can see and change environment/economic/equity weights.
- Every recommended option has score contributions and group benefit values.
- Drought, extreme rainfall, and budget-cut scenarios are separately identifiable and reproducible.
- The decision explanation cites retrieved evidence and repeats the actual saved calculation outputs.
- Missing or uncertain inputs stay visible and do not become invented facts.

Implementation status: requirements-aligned design proposal. Validate costs, indicator definitions, local data coverage, climate assumptions, and social impact measures before presenting outputs as forecasts or policy advice.

14. SDDI CKAN Kubernetes chart mapped to AWS

The linked SDDI chart is the deployment layer for the linked CKAN Docker images. It deploys a self-contained CKAN catalog with PostgreSQL/PostGIS, Solr, Redis, and CKAN Datapusher. On AWS, Amazon EKS is the direct Kubernetes target. CKAN supplies catalog, discovery, and geospatial metadata capabilities; it does not supply the intervention optimizer, climate simulator, or sustainability dashboard.

```
USERS
  | Route 53 + ACM TLS
  | ALB / AWS Load Balancer Controller
  +-- CloudFront + S3: React + TypeScript + MapLibre
  +-- EKS private subnets: SDDI Helm chart
      |-- CKAN API and web catalog
      |-- PostgreSQL + PostGIS (persistent volume)
      |-- Solr + Redis
      +-- Datapusher -> CKAN DataStore
  App API: FastAPI on ECS Fargate or EKS
      |-- CKAN REST API adapter
      |-- Lambda for quick optimizer/scenario runs
      |-- SQS + ECS/Batch worker for longer jobs
      |-- DynamoDB run status + S3 result artifacts
      +-- optional Bedrock + OpenSearch explanation
  Data: S3 raw/curated -> Glue -> Athena
  Identity/security/ops: Cognito, IAM/IRSA, Secrets, KMS,
                         CloudWatch, CloudTrail, budgets
```

CKAN fit and boundaries

Use CKAN to publish datasets with owner, topic, spatial/temporal coverage, license, lineage, and resource links. The CKAN API powers dataset search and metadata in the app. PostGIS supports CKAN spatial catalog capabilities; Solr supports search; Redis supports CKAN service functions; Datapusher loads supported tabular resources into CKAN DataStore. Datapusher is not the Glue ETL pipeline. Keep analytics in curated S3/Athena and optimization in your application backend.

Storage and ingress decisions

Use EKS persistent volumes, typically EBS through a StorageClass, for chart-managed state. Store raw, curated, and scenario files in S3. The chart component list alone does not establish that CKAN resource uploads are configured for S3; verify a compatible extension and IAM integration before relying on that. Keep EKS nodes and stateful services in private subnets. Expose CKAN and the frontend via HTTPS through ALB/ingress, with Route 53 DNS and ACM TLS; ingress-nginx/cert-manager are alternatives. Use IRSA for least-privilege pod access.

Before public exposure

Change the chart documentation default admin password, pin stable chart/image releases, restrict admin access, enable persistent storage, and test PostgreSQL/PostGIS backup and restore. Do not deploy debug/devel images.

15. Detailed frontend and backend technology stack

Keep the public product UI separate from CKAN administration. The React app calls your application API; the API brokers CKAN metadata and coordinates analysis. This prevents coupling the user experience to CKAN templates and keeps decision calculations outside the catalog.

Frontend stack

Layer	Suggested technology	Purpose
Web application	React + TypeScript + Vite	Typed dashboard and scenario builder.
Map	MapLibre GL JS; GeoJSON/vector tiles	Neighborhood boundaries, risks, and intervention overlays. Amazon Location Service is optional for basemaps/geocoding; verify region and tile licensing.
Charts/forms	Recharts or ECharts; React Hook Form + Zod	Baseline/scenario charts and accessible validation.
Hosting	S3 + CloudFront; Amplify Hosting alternative	Static site delivery over HTTPS, separate from CKAN pods.
Identity	Amazon Cognito (optional for MVP)	App user authentication; CKAN roles need deliberate integration.

Backend stack

Component	Suggested technology / AWS	Responsibility
API	Python FastAPI on ECS Fargate; API Gateway if Lambda-oriented	/areas, /catalog, /runs, /runs/{id}; validate identity and inputs.
Catalog adapter	CKAN REST API client	Search datasets and map metadata to app contracts.
Data preparation	S3 + Glue Data Catalog/ETL + Athena; GeoPandas/Shapely jobs as needed	Validate, reproject, aggregate by neighborhood, preserve provenance.
Optimization	Python deterministic scoring; Lambda short runs, ECS/Batch longer runs	Budget/land constraints, weighted objective, Pareto options, uncertainty.
Run state	DynamoDB metadata/status; S3 snapshots and outputs	Immutable run IDs with versioned inputs and results.
AI explanation	Bedrock; OpenSearch only when retrieval corpus warrants it	Phrase saved outputs using approved evidence; optional for MVP.

Hackathon-sized default

React + TypeScript + MapLibre frontend; CKAN chart on EKS; FastAPI on ECS Fargate; S3/Glue/Athena for data; deterministic Python scoring; DynamoDB + S3 for reproducible runs. Add Cognito, Bedrock, OpenSearch, Batch, or SageMaker only when the demo needs them.

16. End-to-end backend flow and API

Return a run ID quickly, show progress for longer jobs, and compare immutable saved results.

When a planner runs scenarios

1. Frontend: validate area, weights, interventions, and budget; send a request with baseline version.
2. API: authenticate and validate; resolve dataset identifiers through CKAN; load versioned baseline data from curated S3/Athena.
3. Planner: calculate feasible packages and shocks. For longer work, publish a job to SQS and run an ECS/Batch worker; for a demo, invoke the calculation directly.
4. Persistence: store request/status in DynamoDB and large outputs in S3; retain CKAN resource IDs, dataset versions, assumptions, and model version.
5. Explanation: optionally send saved score breakdown and approved evidence to Bedrock; validate response and keep evidence references.
6. UI: poll status, draw map overlays/charts, compare results, export from the saved run.

Example API shape

```
POST /api/v1/runs
Authorization: Bearer <Cognito JWT>
{
  "area_id": "ward-12-demo",
  "baseline_version": "2026-10-demo-1",
  "budget_inr": 25000000,
  "land_limit_m2": 12000,
  "weights": {"environment": 0.45, "economic": 0.25, "equity": 0.30},
  "shocks": ["drought", "extreme_rain", "budget_cut_20"]
}

202 Accepted -> {"run_id":"run_01...", "status":"queued"}
GET /api/v1/runs/run_01... -> immutable result + sources
```

Never put AWS credentials, CKAN admin passwords, or Bedrock permissions in browser code. Use short-lived authenticated tokens. A simple API + SQS worker is enough; Step Functions is optional for multi-stage workflows.

17. AWS services that complete the architecture

The original list covered analytics and AI but omitted web delivery, networking, identity, secrets, monitoring, recovery, and cost controls. This is the full reference architecture; implement only what the hackathon needs.

Delivery and networking

Route 53: DNS. **ACM:** TLS. **CloudFront + S3:** static frontend. **VPC:** subnet/routing/security boundary. **ALB / AWS Load Balancer Controller:** ingress to EKS/API. NAT Gateway only where private workloads need outbound internet; monitor cost.

Compute and orchestration

EKS: SDDI Helm chart. **ECR:** private container images. **ECS Fargate:** API/workers without adding all services to Kubernetes. **Lambda:** short event-driven runs. **SQS:** durable long-job queue. **Step Functions:** optional multi-step orchestration. **EventBridge:** scheduled refreshes. **AWS Batch:** heavier queued search/simulation.

Data and geospatial

S3: raw, curated, exports, artifacts. **Glue:** catalog/schema and ETL. **Athena:** SQL over curated S3. **PostGIS:** CKAN spatial support. **Location Service:** optional map-related APIs, not city data or map rendering. **OpenSearch:** only with a justified retrieval/search corpus.

Identity, security, operations

Cognito: app sign-in. **IAM + IRSA:** least-privilege EKS identity. **Secrets Manager or SSM Parameter Store:** secrets/config. **KMS:** encryption. **WAF:** optional edge filtering. **CloudTrail:** AWS API audit. **CloudWatch:** logs, metrics, alarms.

Backup and cost

Snapshots/AWS Backup: recovery for persistent data where supported. **AWS Budgets and Cost Anomaly Detection:** monitor EKS, NAT, OpenSearch, and simulation spend. **QuickSight:** optional analyst BI, not necessarily the product UI. **Managed Grafana:** operational metrics, not the sustainability dashboard.

Data path

```
Sources -> S3/raw -> Glue validation/transforms
-> S3/curated (Parquet/GeoParquet) -> Athena indicators
-> CKAN catalog metadata + resource links
-> FastAPI reads baseline + CKAN provenance
-> Lambda/ECS calculates packages and shocks
-> DynamoDB run status + S3 output artifacts
-> React map/dashboard + optional Bedrock explanation
```

18. Deployment plan, security, and reference links

Separate the target design from the demo footprint. A complete production setup is too much to build before a hackathon.

Suggested build sequence

1. Run the chart locally using its Docker Desktop or Minikube example. Load one sample dataset and verify CKAN search/resource retrieval.
2. Deploy CKAN to EKS only if cluster setup is achievable; otherwise demo local CKAN and show the EKS target architecture.
3. Store curated files/results in S3. Start with one FastAPI service and deterministic Python optimizer; add Glue/Athena for repeatable preparation.
4. Host React on S3/CloudFront and build the map/comparison. Add Cognito only if login matters to the demo.
5. Add Bedrock once outputs and evidence are stable. Add SageMaker only for a supported task with suitable data.

Security checklist

Change the chart documented default admin credential before exposure; pin stable releases; restrict admin access and keep database/search private; use persistent volumes and test backups; store secrets in Secrets Manager/SSM; use IRSA, KMS, HTTPS, CloudWatch and cost alerts. Do not expose debug images or assume CKAN automatically shares Cognito identity.

Key boundaries

CKAN DataStore is not the analytics warehouse. Datapusher imports supported files into CKAN DataStore; Glue/Athena + S3 are the analysis path.
The chart component list does not prove S3 resource storage is configured. Verify extension and IAM integration before relying on CKAN uploads in S3.
Location Service is not the frontend map renderer. The app still needs MapLibre/another library and an allowed tile source.
Bedrock is not the optimizer. Deterministic code/validated models calculate; Bedrock can explain saved results.

References

TUM-GIS SDDI CKAN Kubernetes Helm chart
TUM-GIS SDDI CKAN Docker images
Amazon EKS | Amazon S3 | AWS Glue
Amazon Athena | Amazon Location Service | Amazon Bedrock
AWS Lambda | Amazon Cognito | Amazon CloudWatch

This is an implementation proposal, not proof of a deployed system. Confirm chart values, storage integration, pinned versions, regional availability, costs, and local-data assumptions before deployment or presenting estimates as forecasts.